

Chapter 11

Inheritance

Object-oriented TypeScript, test first

Animals

Add a catAdd a dogEverybody speak

Tom says Meow · Rex says Woof · Felix says Meow · Bella says Woof

Tom, a cat

Speak

Rex, a dog

Speak

Felix, a cat

Speak

Bella, a dog

Speak

Today

- `extends` , and what a subclass inherits
- `abstract` classes and methods
- `override` - a keyword, and compulsory
- `protected`
- `super(...)` and `super.method()`
- no `final` - what to do instead
- **one test suite for every subclass**

Is-a or has-a?

- "a cat **is an** animal" - inheritance
- "a student **is a** person" - inheritance
- "a car **has an** engine" - composition (Chapter 9)
- say it out loud: if it sounds wrong, don't inherit
- inheritance: write what they share **once**

cat1 to cat11, in one slide

Java	TypeScript
<code>class Cat extends Animal</code>	the same
<code>@Override</code> (optional)	<code>override</code> (required)
<code>Animal a = new Cat()</code>	<code>const a: Animal = new Cat("Tom")</code>
<code>abstract class</code> , <code>method</code>	the same
<code>final</code> class, <code>method</code>	no <code>final</code>

An abstract superclass

```
export abstract class Animal {  
  constructor(protected readonly name: string) {}  
  
  public abstract getSound(): string;  
  
  public speak(): string {  
    return `${this.name} says ${this.getSound()}`;  
  }  
  
  public toString(): string {  
    return `${this.name}, an animal`;  
  }  
}
```

A subclass

```
export class Cat extends Animal {  
  public override getSound(): string {  
    return "Meow";  
  }  
  
  public override toString(): string {  
    return `${this.name}, a cat`;  
  }  
}
```

- no constructor: `new Cat("Tom")` uses **Animal's**
- Java never inherits constructors - TypeScript does

Abstract: the compiler holds you to it

```
new Animal("Thing")
```

TS2511 Cannot create an instance of an abstract class.

```
class Cow extends Animal {}
```

TS2515 Non-abstract class 'Cow' does not implement inherited abstract member getSound from class 'Animal'.

override and noImplicitOverride

`toString()` without `override`:

TS4114 This member must have an 'override' modifier because it overrides a member in the base class 'Animal'.

`override getSound()`:

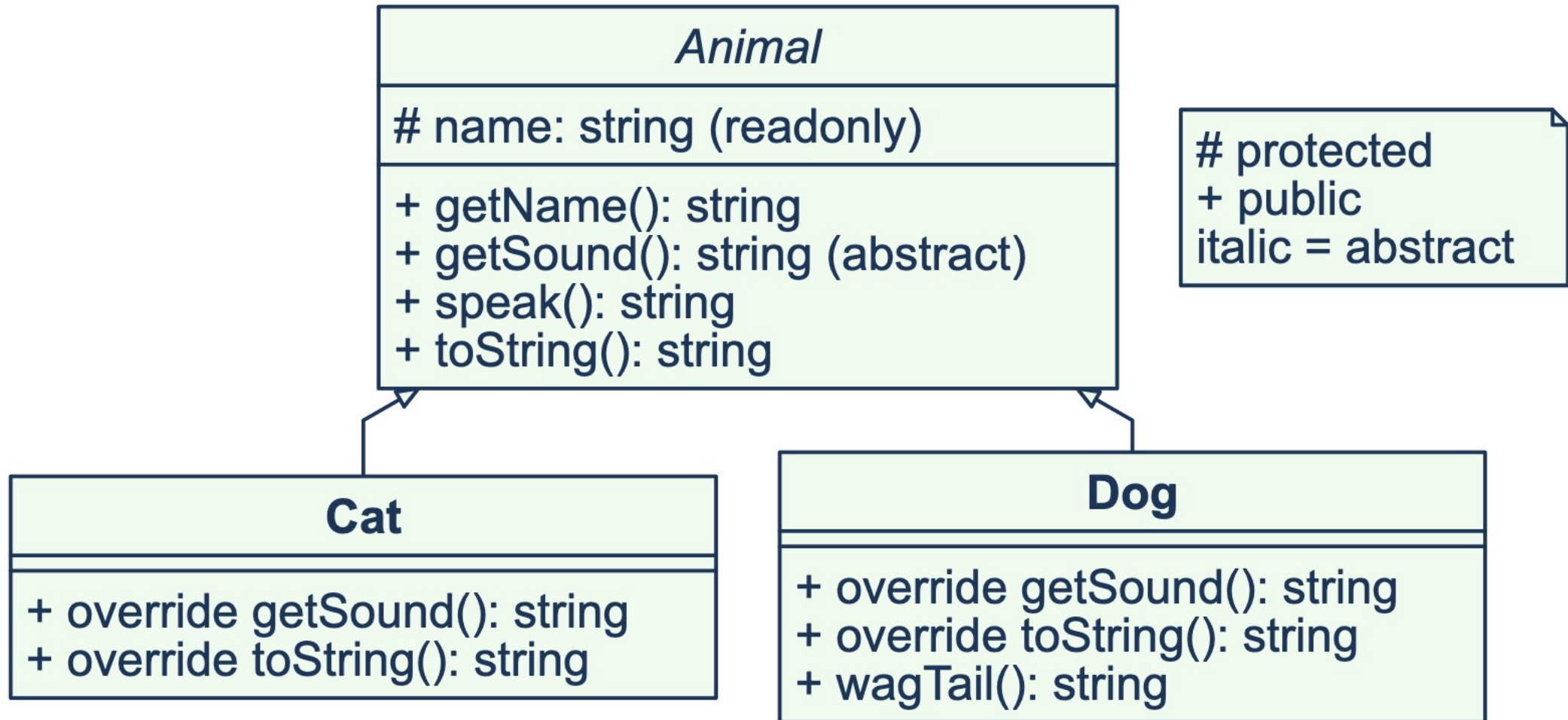
TS4117 This member cannot have an 'override' modifier because it is not declared in the base class 'Animal'. Did you mean 'getSound'?

Adding something new

```
export class Dog extends Animal {  
  // ... getSound, toString  
  public wagTail(): string {  
    return `${this.name} wags their tail`;  
  }  
}  
  
const pet: Animal = new Dog("Rex");  
pet.wagTail();
```

TS2339 Property 'wagTail' does not exist on type 'Animal'.

The hierarchy



A cat is an animal

```
const animals: Animal[] = [new Cat("Tom"), new Dog("Rex")];  
  
export const chorus = (animals: Animal[]): string[] =>  
  animals.map((animal) => animal.speak());  
// ["Tom says Meow", "Rex says Woof"]  
  
tom instanceof Animal // true - classes exist at run time
```

- the **object** decides which method runs
- the **variable's type** decides which methods exist
- Book 2 starts here: polymorphism

toString

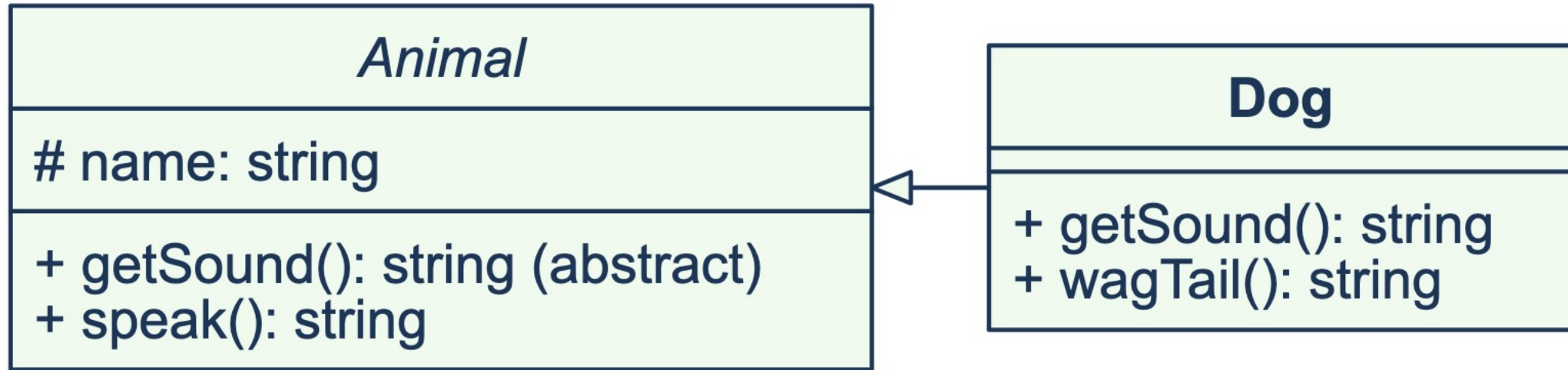
- `${animal}` calls `animal.toString()` - the subclass's
- no `toString()` at all: `[object Object]`
- Java's default: `Animal@1b6d3586`

protected

	class	subclasses	everyone else
private	yes	no	no
protected	yes	yes	no
public	yes	yes	yes

- no package access (TypeScript has no packages)
- start `private` ; open up only when a subclass needs it

Exercise 11.1 - which lines compile?



```
const pet: Animal = new Dog("Rex");
pet.speak();           // 1
pet.wagTail();         // 2
pet.name;              // 3
new Animal("Thing");   // 4
```

Person: an ordinary superclass

```
export class Person {  
  constructor(  
    protected readonly name: string,  
    protected readonly email: string,  
  ) {  
    if (!email.includes("@")) {  
      throw new Error(`Not an email address: ${email}`);  
    }  
  }  
  // getName, getEmail, getRole, toString  
}
```

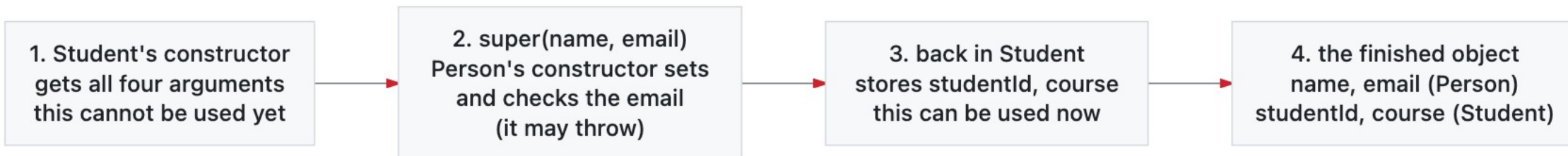
super(...)

```
export class Student extends Person {  
  constructor(  
    name: string,  
    email: string,  
    private readonly studentId: string,  
    private readonly course: string,  
  ) {  
    super(name, email);  
  }  
}
```

- `name`, `email`: plain parameters, **passed on**
- `studentId`, `course`: Student's own

What new Student(...) does

```
new Student("Ann", "ann@college.ie", "S1", "Computing")
```



```
toString() : super.toString() gives "Ann <ann@college.ie>" and Student adds  
", student S1, Computing"
```

The rules, enforced

TS2377 Constructors for derived classes must contain a 'super' call.

TS17009 'super' must be called before accessing 'this' in the constructor of a derived class.

TS2554 Expected 2 arguments, but got 1.

- every Student goes through Person's email check

super.toString()

```
public override toString(): string {  
    return `${super.toString()}, student ${this.studentId}`  
        + `, ${this.course}`;  
}  
// "Ann <ann@college.ie>, student S1, Computing"
```

- build on the superclass's version - don't repeat it

BankAccount: the rules, once

```
export abstract class BankAccount {  
  protected balance: number = 0;  
  
  public withdraw(amount: number): void {  
    this.requirePositive(amount);  
    if (amount > this.available()) {  
      throw new Error(`Not enough money: ...`);  
    }  
    this.balance -= amount;  
  }  
  
  public abstract available(): number;  
}
```

The overdraft, test first: red

```
const account = new CurrentAccount("Ann", 100);  
account.deposit(50);  
account.withdraw(120);  
assertEquals(account.getBalance(), -70);
```

```
not ok 10 - a current account can withdraw into its overdraft  
  Error: Not enough money: €50.00 available  
    at CurrentAccount.withdraw (src/BankAccount.ts:35:13)  
    at tests/current.test.ts:12:11
```

Green

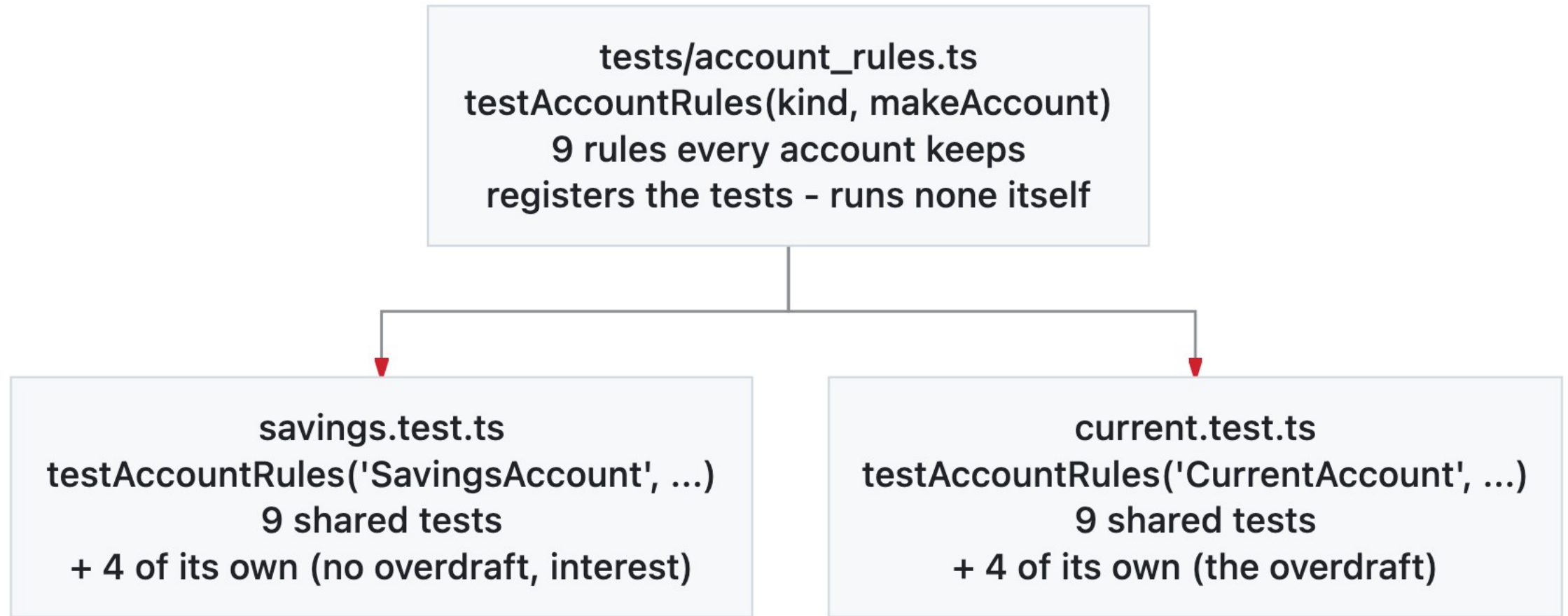
```
public override available(): number {  
    return this.balance + this.overdraftLimit;  
}
```

- thrown in `BankAccount.ts`, but the bug was in `available()`
- read the stack: where it was thrown, then who called it

One test suite, every subclass

```
export const testAccountRules = (  
  kind: string,  
  makeAccount: (owner: string) => BankAccount,  
): void => {  
  Deno.test(`${kind}: a new account is empty`, () => {  
    assertEquals(makeAccount("Ann").getBalance(), 0);  
  });  
  // ... eight more rules  
};  
  
testAccountRules("SavingsAccount", (o) => new SavingsAccount(o));
```

Registered once per subclass



- any `BankAccount` must keep the rules (Liskov, the L in SOLID)

No final

Java	TypeScript
<code>final</code> field	<code>readonly</code>
<code>final</code> class	<code>private constructor</code> + static factory
<code>final</code> method	nothing

For methods: private helpers, a comment, **shared tests**, shallow hierarchies – and composition (Book 3)

Abstract class or interface?

	interface	abstract class
method bodies, fields	no	yes
how many	any number	one
instanceof	no	yes

- only a shape? **interface**
- shared code or state? **abstract class**

Summary

- `extends` for is-a; `abstract` for "not on its own"
- `override` on every overriding method
- `protected` : subclasses only - start `private`
- `super(...)` before `this` ; `super.method()` to build on
- no `final` : `readonly` , private constructor, tests
- shared rules: a function that registers tests

What next: Book 2

Object-oriented TypeScript, further

- polymorphism, narrowing, discriminated unions
- generics, `Map` and `Set`, `null` and `undefined`
- errors, test doubles
- code quality: static and runtime analysis